Contents

How much delay is safe? A plain guide to the retransmission limit

Written 2026-09-10. Audience: the person who has to pick `D_A` and the configured `CLRT_new` for a real deployment and defend the choice afterwards.

The mechanism works by holding packets. Holding a packet is only safe up to a point, and past that point somebody sends the packet again. This note explains where that point is, why there are **two** of them and not one, how the limit is decided in general, and what we actually measured on our own testbed.

Companion documents: `TIMEOUT_AND_RETRANSMISSION_AUDIT.md` (the measurements this note summarizes), `audit_current/RESPONSE_LATENCY_BUDGET.md` (the search for a standards limit), `NOTATION_MAPPING.md` (the symbols).

A PDF of this note sits beside it. Regenerate it after any edit with

```
tools/render_md_pdf.sh HOW_MUCH_DELAY_IS_SAFE.md
```

---

1. The one idea

A retransmission is not caused by delay. It is caused by **a sender running out of patience**.

Every TCP sender keeps a stopwatch on the bytes it has sent but not yet had acknowledged. When the stopwatch passes a threshold, the sender assumes the bytes were lost and sends them again. That threshold is the **retransmission timeout**, or **RTO**.

So to find the limit, you never ask ”how long may I delay a packet”. You ask three questions about each packet you touch:

1. **Whose stopwatch does this packet stop?** — Which machine is waiting for it.
2. **What is on that stopwatch?** — Which of that machine's bytes are unacknowledged.
3. **How long will that machine wait?** — What its RTO is.

Delay the packet longer than the answer to (3), and the machine in (1) resends the bytes in (2).

---

2. There are two limits, because there are two senders

Our mechanism holds two packets, both travelling from the outstation to the master. They stop two different stopwatches on two different machines.

2.1 Holding the acknowledgment — this pressures the master

The master sends a DNP3 read request. That request is TCP *data*, so the master starts a stopwatch on it. The outstation's TCP stack replies almost immediately with a bare acknowledgment, and that acknowledgment is what stops the master's stopwatch.

We hold that acknowledgment for `D_A`.

> **If `D_A` is too large, the master resends its request**, because from the master's point of view its request went unanswered.

The exposure is easy to state, because it is the whole interval from the master pressing send to the acknowledgment coming back:

```
master exposure  =  D_A  +  the outstation's own acknowledgment latency  +  the round-trip path
```

Everything except `D_A` is small and outside your control. `D_A` is the term you set.

A useful fact: holding a bare acknowledgment can never make the *outstation* resend anything. TCP only retransmits segments that consume sequence space — data, `SYN`, `FIN`. A bare acknowledgment carries no data, so no stopwatch runs on it at either end. Holding it is one-sided pressure on the master alone.

2.2 Holding the response — this pressures the outstation

The outstation's application then produces the response. That response is TCP *data*, so now the **outstation** starts a stopwatch on it. What stops that stopwatch is the master's acknowledgment of the response — and the master cannot send that acknowledgment until we release the response.

> **If the response is held too long, the outstation resends its response**, because from the outstation's point of view its answer went unacknowledged.

```
outstation exposure  =  the response hold  +  the return path
                        +  the master's delayed-acknowledgment delay
```

The response hold is `e_R − t_R`: from the response arriving at the switch to it leaving. You do not set it directly — the schedule sets it — but it has a hard ceiling you *do* set:

```
                e_R = t_A + D_A + CLRT_new  =  t_A + D
and             t_R > t_A            (the response always arrives after the acknowledgment)
therefore       response hold  =  e_R − t_R  <  D
```

The response hold can never exceed the release budget `D = D_A + CLRT_new`. That single number bounds the outstation-side exposure. On our campaign setting, `D_A` = 20 ms and the configured `CLRT_new` = 4 ms, so `D` = 24 ms, and the response is never held longer than 24 ms — in practice about 22 ms, because the outstation's own interval `CLRT_original` (median 2.1 ms) is subtracted from it.

2.3 The two limits side by side

	master side	**outstation side**
which packet we hold	the bare acknowledgment	the application response
whose stopwatch it stops	the master's, on its request	the outstation's, on its response
what you set	`D_A` directly	`D = D_A + CLRT_new`, as a ceiling
what else is in the exposure	the outstation's acknowledgment latency, the path	the return path, the master's delayed-ACK delay
what happens if you overrun	the master resends the **request**	the outstation resends the **response**
can you see it in a master-side capture?	**yes**, a duplicate request	**no** — see §7

	master side	outstation side
do we know the limit for our testbed?	approximately: read it off the master	**no**, never read back

The two are not symmetric, and the second one is the harder one. That is the honest headline of this note.

---

3. How the waiting time is decided, in general

The rule is RFC 6298, and it is short enough to state fully.

Before the connection has any round-trip measurement, the sender uses a fixed initial value:

```
RTO = 1 second
```

After the first round-trip measurement `R`:

```
SRTT   = R                          (smoothed round-trip time)
RTTVAR = R / 2                      (round-trip variation)
RTO    = SRTT + max(G, 4 * RTTVAR)  (G is the kernel's clock granularity)
```

After every later measurement `R'`:

```
RTTVAR = (3/4) * RTTVAR + (1/4) * | SRTT − R' |
SRTT   = (7/8) * SRTT   + (1/8) * R'
RTO    = SRTT + max(G, 4 * RTTVAR)
```

Read in words: **the timer is the typical round trip, plus four times how much the round trip has been jumping about.** A path that is slow but steady gets a tight timer. A path that is fast but erratic gets a loose one.

Then three clamps:

- **A floor.** RFC 6298 §2.4 says the result should be rounded up to 1 second. Mainstream kernels do not obey this; Linux clamps at its compiled-in `TCP_RTO_MIN` of 200 ms. This is the single most important number in the whole note, and §4 explains why.
- **A ceiling.** An implementation may cap the RTO, provided the cap is at least 60 seconds.
- **Backoff.** Each time the timer fires, the sender doubles it and tries again: 200 ms, 400 ms, 800 ms, and so on. So one overrun does not just cost one duplicate; it costs a doubling.

One more rule that matters here — Karn's algorithm. A sender must not take a round-trip sample from a segment that was retransmitted, because it cannot tell which copy was acknowledged. So once you provoke a retransmission, the timer stops learning from that exchange, and it stays inflated until clean exchanges resume.

---

4. The trap: you cannot budget against the timer you will end up with

Here is the mistake that looks clever and is wrong.

Because the hold becomes part of the round trip the master measures, `SRTT` rises to include it, so `RTO` rises too. It is tempting to conclude that a hold is self-accommodating: hold longer, the timer grows, you are always safe.

That is true in the steady state, and it is useless as a budget, for three reasons.

1. **The steady state is not where you start.** Before any measurement the timer is at its initial value, and after enough idle time or enough backoff it is back near the floor. A connection that has just opened, or has just been quiet, has a *small* timer while your hold is already at full size. The first transaction after a quiet period is the dangerous one.

2. **The floor does not adapt.** Whatever `SRTT` says, the kernel will not go below its floor — and the floor is a *lower* bound on the timer, so it protects you. But the floor is also the value you get in the worst case, which is exactly the case you must budget for. Budget against the floor, never against the adapted value.
3. **A variable hold is much worse than a constant one.** Look at the formula again: the variation term is multiplied by **four**. A hold that is the same every time is absorbed into `SRTT` and costs you nothing in `RTTVAR`. A hold that jitters by ±10 ms adds roughly 40 ms of slack — which sounds helpful until you realise the jitter is also what your own mechanism is trying to remove. Our policy holds by a constant, which is the well-behaved case.

So the rule is: budget against the floor, and make the hold constant.

---

5. The procedure

Six steps. Do them in order.

Step 1 — List every packet the mechanism delays, and by how much

For this design there are two, and both bounds are configured values:

packet	held by	bound
the outstation's acknowledgment	`D_A`	exactly `D_A`
the outstation's response	the schedule	strictly less than `D = D_A + CLRT_new`

Step 2 — Name the machine whose timer each one stops

From §2: the acknowledgment stops the **master's** timer; the response stops the **outstation's** timer. Write both down. Skipping the second is the usual failure.

Step 3 — Find each machine's floor

The master. If it is a Linux host, you can simply read it, per socket, while traffic is running:

```
ss -ti
```

Look for `rto:` and `ato:` on the socket's second line. As an illustration of what the command gives you, a live socket on the workstation this note was written on — not the DNP3 master — reports:

```
cubic wscale:8,7 rto:208 rtt:7.78/2.119 ato:40 mss:1412 ...
```

Read it as follows. `rtt:7.78/2.119` is `SRTT`/`RTTVAR` in milliseconds, so the formula in §3 gives about $7.78 + 4 \times 2.119 \approx 16$ ms — far below the floor. `rto:208` is what the kernel is actually using: the floor, give or take a rounding tick. That gap between 16 ms and 208 ms is §4's point made visible. `ato:40` is the delayed-acknowledgment timeout currently in effect, which you will need in Step 4.

Run this on the actual master, against the actual outstation, while the poll loop is running. A socket to some other host tells you about that path, not yours.

Two more things worth knowing about the master:

- The floor is usually not a sysctl. On the Linux 5.15 host used above, `sysctl -a` exposes no `tcp_rto_min` at all: the 200 ms value is compiled in. It can still be overridden **per route** — `ip route change <dest> ... rto_min 50ms` — so run `ip route show` and check for such an override before trusting the default. Somebody may have lowered it. Newer kernels have begun exposing it as a sysctl; check your own kernel rather than assuming either way.

- `net.ipv4.tcp_retries2` (default 15) decides how many doublings happen before the connection is abandoned, not when the first retransmission happens. It is the difference between ”a duplicate” and ”the session dies”.

The outstation. You almost certainly cannot read it. It is a closed embedded stack in a protection relay, its TCP parameters are not in the settings list, and the vendor manual does not state them. This is the real gap, and there are only two honest ways to close it:

- **Ask the vendor** for the stack's initial RTO and minimum RTO, in writing; or
- **Measure it.** Put a tap on the relay-facing link, raise the response hold in steps, and find the hold at which duplicate response segments first appear. The first duplicate marks the floor. Then set your operating point well below it.

Until you have done one of those, treat the outstation-side limit as **unknown**, and say so. Do not substitute the master's number for it.

Step 4 — Subtract everything on the path that is not your hold

Your hold is not the only thing consuming the budget.

```
master side:      budget_A  =  RTO_floor(master)
                              − the outstation's own acknowledgment latency
                              − the round-trip path delay
                              − the mechanism's own release jitter

outstation side:  budget_D  =  RTO_floor(outstation)
                              − the return path delay
                              − the master's delayed-acknowledgment delay   (the ato: value)
                              − the mechanism's own release jitter
```

The delayed-acknowledgment term is the one people forget. The master's TCP does not always acknowledge data the instant it arrives; it may wait, so that the acknowledgment can ride along with the master's next request. RFC 1122 allows up to 500 ms of this; real stacks use far less (40 ms on the host above). It sits directly on top of your response hold in the outstation's exposure, so it comes out of the outstation's budget, not the master's.

Step 5 — Apply a safety factor, and pick the smaller of the two

Do not spend the whole budget. A reasonable rule is to stay at or below **a quarter** of the smaller floor, which leaves room for the terms you measured badly and for the ones you did not think of.

```
D_A  ≤  budget_A / 4          and          D  ≤  budget_D / 4
```

Then apply the binding one. With a 200 ms floor and a quarter-margin, that is $D \leq 50$ ms — and in our system the mechanism's own ceiling (§6) is tighter than that anyway, which is the comfortable case.

Step 6 — Verify by counting, not by arguing

A budget calculation is a prediction. Confirm it by running the workload and counting.

```
# duplicates on the master-facing link
tshark -r capture.pcap -Y "tcp.analysis.retransmission" | wc -l

# the same, from the master's own kernel counters, before and after a run
netstat -s | grep -i retrans

# the DNP3 invariant: one response per request, so these two counts must match
tshark -r capture.pcap -Y "dnp3.al.func == 1"   | wc -l    # READ requests
tshark -r capture.pcap -Y "dnp3.al.func == 129" | wc -l    # responses
```

Do not try to spot duplicates by listing `dnp3.al.seq` values and looking for repeats: the DNP3 application sequence number is four bits wide and wraps every sixteen transactions, so repeats are normal. Compare counts, or compare sequence numbers only inside one short window.

Then repeat on a **relay-facing** tap for the outstation side. If you have not tapped the relay-facing link, you have not verified the outstation side — see §7.

---

6. What we measured, and what actually binds

On our testbed at `D_A` = 20 ms, configured `CLRT_new` = 4 ms, so `D` = 24 ms:

quantity	value	source
worst acknowledgment wait, any class, any arm	**29.150 ms**	132 captures, 63,360 exchanges
worst request-to-response latency	**77.713 ms**	same
TCP retransmissions seen, master-facing	**0** in 63,360 exchanges, both arms	independent TCP parser over all captures
application timeouts	**0**	the driver's own unfiltered log
`NO_SELECT` refusals	**0** of 2,640 obfuscated OPERATE exchanges	same
assumed master RTO floor	200 ms (Linux constant, **not read back on that host**)	Linux `TCP_RTO_MIN`
resulting margin	about **6.9×** against 200 ms, about **34×** against RFC's 1 s	arithmetic
outstation RTO floor	**unknown**, never recorded	—

And then the surprise: the transport is not what binds. The mechanism holds a packet by starving its queue with a reservoir of internally generated blocker packets, and that reservoir has a finite pass budget. When the budget runs out the packet is released whatever the deadline says. The control plane translates that budget into a time horizon `H` = **30.8 ms** for our configuration, and refuses to install a policy whose budget exceeds 24.8 ms once the outstation's own latency and a safety margin are subtracted. Measured, the acknowledgment interval tracks `D_A` up to 30 ms and then saturates near 31.07 ms.

So on this build:

```
mechanism ceiling ~31 ms  <<  master RTO floor ~200 ms  <<  RFC floor 1 s
```

The mechanism reaches its own limit about six times before the transport notices — 200 ms against a ceiling near 31 ms, and 6.9× against the worst wait we actually observed. That is a good place to be: the failure you will actually hit is a benign one (the hold stops growing) rather than a protocol one (duplicate packets).

One warning about that ceiling. It is enforced as a count of blocker passes, not as milliseconds. The conversion is `H = B × K / rate`, where `rate` is the line rate of the internal loopback port. At 25G the horizon is 30.8 ms; at 10G the same pass budget becomes about 99 ms. A silent port-speed renegotiation therefore moves the ceiling by a factor of three without anyone changing a setting. Assert the port speed at setup, and re-derive `H` from the model rather than carrying the number forward as a constant.

---

7. Why our own numbers do not settle the outstation side

This is the part to read twice.

Our capture vantage is the master's own interface. That is the only tap. Two consequences:

1. **A retransmission by the outstation would never reach the tap.** The switch program contains a duplicate-suppression rule that drops a response retransmission matching a TCP position it has already seen. If the outstation resent a held response, the switch would absorb it silently. ”Zero response retransmissions” in our table means *no duplicate reached the master*. It does **not** mean the outstation never retransmitted.
2. **The outstation's RTO was never read back**, so we have no floor to compare against even in principle.

Settling it needs one of: a relay-facing tap, or a readback of the switch's suppression counter, or the vendor's stack parameters. All three are listed as open work. Anyone reusing this mechanism should close one of them before claiming the outstation side is safe.

---

8. The other clocks in the room

TCP is not the only thing counting. Take the **smallest floor among all of these**, not just the TCP one.

timer	owner	what it bounds	value on our testbed
TCP RTO	either kernel	unacknowledged bytes	master ≈ 200 ms floor (assumed); outstation unknown
delayed acknowledgment	the receiving kernel	how long an ACK may wait to ride along	40 ms observed on a Linux host; RFC ceiling 500 ms
application receive timeout	the master program	one blocking read	3.0 s, and it is **per read**, not per transaction
application retry policy	the master program	whether a lost request is re-issued	none exists in our drivers
select validity, `STIME01`	the outstation	SELECT accepted → matching OPERATE received	documented default **1.0 s**, range 0.0–30.0 s; the configured value was never read back
data-link timeout, `DTIME01`	the outstation	link-layer confirmation	documented default 1 s, range 0.0–5.0 s
event confirm, `ETIME01`	the outstation	event confirmation	documented default 5 s, range 1–50 s
DNP3 link / application confirm	either endpoint	confirmed frames	not in play; we use unconfirmed user data

Two notes on this table:

- **The select window is the one application-layer timer our mechanism can genuinely violate.** Holding the SELECT's acknowledgment and response delays the master's OPERATE, and if that delay exceeds `STIME01` the OPERATE is refused with `NO_SELECT`. At our setting the worst master-facing SELECT-to-OPERATE gap was 29.968 ms against a documented 1.0 s default, a margin of about 33×, and no refusal ever occurred. But `STIME01` is settable down to 0.0 s, so on a relay configured tightly this becomes the binding limit, not TCP.
- **A per-receive socket timeout is not a transaction deadline.** Ours is 3.0 s, re-armed on every read, so nothing in the master bounded the transaction as a whole. Do not present such a value as a budget.

---

9. What actually goes wrong if you overrun

In rough order of how quickly you notice:

1. **A duplicate request or response appears on the wire.** Wasteful, usually harmless once.
2. **The outstation may answer the duplicate twice.** Now the switch's transaction matching has two responses for one transaction, and its duplicate suppression has to be right. Ours drops a position-matched duplicate; a different design might release both.

3. **The timer doubles.** The next overrun costs 400 ms, then 800 ms. Latency for that connection degrades sharply and stays degraded, because Karn's algorithm stops the timer learning from retransmitted exchanges.
4. **The exchange leaks the very thing you were hiding.** A retransmission is itself a timing event with a characteristic spacing. A defence that provokes retransmissions has replaced one fingerprint with another.
5. **Eventually the connection dies.** After `tcp_retries2` doublings the kernel gives up and tears the connection down, and the master has to reconnect. This is the failure the operator will actually report.

Note that (4) is the reason this is not merely an availability question. Over-holding does not just risk the process; it undermines the defence.

---

10. Worked example

An administrator wants to deploy on a site whose master is Linux and whose relay is unknown.

```
Step 1  Held packets:  acknowledgment for D_A;  response for less than D.

Step 2  Acknowledgment → master's timer.  Response → outstation's timer.

Step 3  Master:      ss -ti on the master, during a poll, shows  rto:208  ato:40.
                     No per-route rto_min override in `ip route show`.
                     Floor = 200 ms.
        Outstation:  not readable.  Treated as UNKNOWN and handled in Step 5.

Step 4  Path round trip measured at 0.4 ms.
        Outstation's own acknowledgment latency measured at 0.6 ms.
        Release jitter of the mechanism, measured spread, 0.7 ms.

        budget_A = 200 − 0.6 − 0.4 − 0.7  =  198.3 ms

Step 5  D_A ≤ 198.3 / 4  =  49.6 ms   → the transport permits D_A up to about 49 ms.
        But the mechanism's own ceiling H is 30.8 ms and the control plane
        enforces an admissible budget of 24.8 ms.
        The mechanism binds.  Choose D = 24 ms, split D_A = 20 ms, CLRT_new = 4 ms.

        Outstation side: D = 24 ms is the ceiling on the response hold.  With the
        floor unknown, this is recorded as an assumption to be discharged by
        the relay-facing measurement in Step 6, not as a verified margin.

Step 6  Run the workload.  Count retransmissions on the master-facing link: expect 0.
        Count duplicate DNP3 application sequence numbers: expect 0.
        Tap the relay-facing link and repeat, to close the outstation side.
        Also confirm STIMEO1 on the relay is not set below, say, 200 ms.
```

The shape of that example is the point: **the transport limit is generous, the mechanism's own limit is tight, and the one number you cannot look up is the outstation's.**

---

11. One-page summary

- A retransmission happens when a **sender** waits too long for proof its bytes arrived. Find the sender, find the proof, find how long it waits.

- Holding the **acknowledgment** pressures the **master**; the exposure is `D_A` plus small terms.
- Holding the **response** pressures the **outstation**; the exposure is bounded by the release budget `D = D_A + CLRT_new` plus the return path plus the master's delayed acknowledgment.
- Holding a **bare acknowledgment** cannot make the outstation retransmit anything — no stopwatch runs on a packet that carries no data.
- The waiting time is `SRTT + 4 × RTTVAR`, clamped by a floor. **Budget against the floor**, not against the adapted value, because a fresh or idle connection sits at the floor.
- **Make the hold constant.** Variation is multiplied by four in the formula, and jitter is what you were trying to remove anyway.
- Read the master's floor with `ss -ti` (`rto:`, `ato:`). You will not be able to read the outstation's — measure it on a relay-facing tap or get it from the vendor.
- Take the **smallest** floor across TCP, the delayed-ACK term, the application timers and the device's own windows, especially the select-to-operate timeout.
- Verify by **counting duplicates**, on both links. A master-side capture cannot see an outstation-side retransmission in this design, because the switch suppresses it.
- On our build the mechanism's own ceiling (~31 ms) binds about six times before the master's assumed transport floor (~200 ms) does, and we measured zero retransmissions in 63,360 exchanges.